Licence Informatique - Parcours MIAGE

SAE 4 - Réaliser une application informatique complète

Professeurs: Yu LI && Frédéric Fürst

Rapport Final

Livré le 29/03/2026

Henry Andrew MEDINA CONDO
Yassir ELMOUDNI

Semestre 5

"L'étudiant déclare avoir réalisé ce travail conformément au règlement de l'Université Picardie Jules Verne."

Table des Matières

1 Introduction..... 3
2 Analyse des écarts.....3
2.1 Objectifs Techniques..... 3
A. Conception de la structure et du modèle..... 3
B. Architecture et Technologies..... 3
C. Efficacité et Optimisation (Points 6 et 7)..... 3
D. Stratégie de Tests..... 4
E. Documentation et Contraintes:.....4
2.2 Objectifs Organisationnels..... 4
A. Respect du planning et des jalons..... 4
B. Respect des contraintes organisationnelles et techniques..... 4
C. Contraintes Juridiques et Éthiques..... 5
2.3 Analyse du projet.....5
A. Facteurs de réussite et gestion des échecs.....5
B. Anticipation des risques et contraintes.....5
C. Adéquation des méthodes et outils.....6
3. Retour d'expérience.....6
3.1. Ingénierie des données et Efficacité.....6
3.2. Architecture et Robustesse..... 6
3.3. Validation et Qualité Logicielle.....7
4. Conclusion.....7
4.1. Synthèse de la Réussite Technique..... 7
4.2. Perspectives et Évolutivité..... 7
Annexes..... 7

1 Introduction

Ce document présente le bilan final du projet Snake AI A* et Apprentissage par Renforcement, réalisé dans le cadre de la SAE 4. Il a pour but d'analyser la transition entre la phase de conception initiale et la réalisation technique achevée au 29 mars 2026. Le rapport détaille les écarts constatés, évalue l'atteinte des objectifs et synthétise le retour d'expérience des développeurs sur l'architecture logicielle et les performances des agents d'IA.

2 Analyse des écarts

2.1 Objectifs Techniques

A. Conception de la structure et du modèle

- **Structure de données adaptée** : Le choix s'est porté sur un vecteur d'état compressé de 11 bits. Cette structure est optimale car elle transforme un environnement complexe en une clé d'indexation simple pour la Q-Table.
- **Modèle fonctionnel** : L'architecture hybride (A* pour l'heuristique et Q-Learning pour l'apprentissage) répond strictement au cahier des charges.
- **Avantages/Inconvénients** :
  - Avantage: La structure tabulaire garantit une interprétabilité totale des décisions de l'IA.
  - Inconvénient: Elle reste limitée à des espaces d'états discrets et ne peut pas généraliser à des grilles infinies sans réentraînement.
- **Combinaison outils/données** : L'intégration de la structure 11-bits avec l'outil de traitement **MsgPack** permet une persistance rapide et efficace des connaissances acquises.

B. Architecture et Technologies

- **Architecture logicielle**: L'application suit un modèle MVC (Modèle-Vue-Contrôleur) strict. Le moteur (*engine*) est totalement indépendant de l'interface (ui), facilitant les tests en mode "headless".
- **Design Pattern Strategy**: L'utilisation de ce pattern pour les agents permet de changer d'algorithme (A* vers RL) sans modifier le contrôleur.
- **Technologies**: Le choix de **Python 3.12** et du gestionnaire **uv** a permis de stabiliser l'environnement de développement et de garantir la portabilité du projet.

C. Efficacité et Optimisation (Points 6 et 7)

- **Occupation mémoire**: L'utilisation du format binaire **MsgPack** au lieu du JSON a réduit l'empreinte disque de la Q-Table de près de 40%, tout en accélérant les temps de lecture.
- **Temps de calcul**: Le calcul du prochain mouvement par l'agent RL s'effectue en temps constant $O(1)$ grâce à l'accès direct par index, dépassant largement les performances de calcul en temps réel de l'algorithme A* sur des grilles denses.

D. Stratégie de Tests

- **Tests fonctionnels**: Une suite de tests sous **Pytest** a été conçue pour valider les collisions, la croissance du serpent et la génération de la nourriture.
- **Correction des écarts** : Les tests ont révélé des réinitialisations incorrectes du GameState lors des duels multithreadés ; ces écarts ont été corrigés par une synchronisation des instances dans le DuelManager.

E. Documentation et Contraintes:

- **Documentation utilisateur** : Un fichier README détaillé et un Makefile ont été produits, permettant à l'utilisateur de lancer l'entraînement (make train) ou le benchmark (make benchmark) en une seule commande.
- **Contraintes techniques** : La principale contrainte identifiée a été la gestion du parallélisme pour les duels d'IA, résolue par l'implémentation d'un ThreadPoolExecutor.

2.2 Objectifs Organisationnels

A. Respect du planning et des jalons

Le calendrier global a été scrupuleusement tenu, avec une transition fluide entre les deux phases majeures :

- **Phase de Conception (Février 2026)** : Livraison du rapport de conception et des modèles UML à la date prévue. Cette étape a permis de fixer les bases théoriques sans retard.
- **Phase de Réalisation (Mars 2026)** : Le développement s'est achevé dans les délais pour la livraison finale du 29 mars. L'utilisation de la méthode **Iterative Development** a permis d'intégrer les fonctionnalités (moteur, puis A*, puis RL) de manière progressive.
- **Benchmark Final** : Les tests de performance ont été réalisés durant la dernière semaine, validant les indicateurs clés (KPI) juste avant la clôture du projet.

B. Respect des contraintes organisationnelles et techniques

Le projet a respecté l'ensemble des contraintes académiques et professionnelles liées au parcours MIAGE :

- **Standardisation du Workflow** : L'introduction d'outils comme **uv** pour la gestion des dépendances et **Makefile** pour l'automatisation a garanti que le projet puisse être exécuté par n'importe quel évaluateur sans conflit d'environnement (Contrainte de portabilité).
- **Collaboration et Versioning** : L'utilisation de **Git** (GitHub) a permis une gestion rigoureuse des versions et une fusion efficace des modules développés séparément (Agent vs Engine).
- **Conformité logicielle** : Le code respecte les normes de nommage Python (PEP 8) et les principes de conception logicielle **SOLID**, assurant une documentation technique lisible et maintenable.

C. Contraintes Juridiques et Éthiques

- **Propriété Intellectuelle** : Le projet utilise des bibliothèques Open Source (Pygame, MsgPack, Pytest). Les licences correspondantes ont été respectées.
- **Intégrité Académique** : Conformément à la déclaration de l'étudiant en page de garde, tous les algorithmes (notamment l'encodage 11-bits et la logique A*) ont été conçus et implémentés par l'équipe, en s'appuyant sur les références bibliographiques citées (Russell & Norvig).

2.3 Analyse du projet

A. Facteurs de réussite et gestion des échecs

- **Réussite majeure : La modularité.** L'utilisation du **Design Pattern Strategy** a été le facteur clé de succès. Elle a permis de développer, tester et comparer les agents A* et RL de manière totalement indépendante, sans aucune régression sur le moteur de jeu.
- **Gestion des échecs (Pivot technique) :** Initialement, le format JSON ralentissait les cycles de tests à cause de la taille de la Q-Table. L'échec de la solution JSON a été transformé en réussite par la migration vers **MsgPack**, augmentant drastiquement les performances d'I/O.
- **Complexité du multi-threading :** La mise en place du DuelManager a présenté des difficultés de synchronisation. La réussite est venue de l'isolation des instances de SnakeEngine pour éviter tout partage de mémoire non contrôlé.

B. Anticipation des risques et contraintes

- **Risque d'explosion combinatoire :** Ce risque, identifié dès le rapport de conception, a été parfaitement anticipé par l'**encodage 11-bits**. Sans cette

réduction de l'espace d'états, le Q-Learning n'aurait jamais convergé dans les délais impartis.
- **Contrainte de performance :** L'anticipation du besoin de "Headless Mode" (exécution sans interface graphique) a été cruciale pour réaliser les 20 000 épisodes d'entraînement nécessaires à la convergence du modèle RL.
- **Dépendances logicielles :** L'utilisation de `uv.lock` a permis d'anticiper les risques de conflit de versions entre les environnements de développement des différents membres de l'équipe.

C. Adéquation des méthodes et outils

- **Python 3.12 et Pygame :** Outils parfaitement adaptés pour un prototypage rapide. Pygame, bien que limité pour des jeux complexes, s'est avéré idéal pour manipuler une grille de 20x20 avec une latence minimale.
- **TensorBoard :** Cet outil a été indispensable pour monitorer la perte (*Loss*) et les récompenses en temps réel, transformant une "boîte noire" (l'apprentissage) en un processus visuel et analysable.
- **Le Makefile :** Bien que simple, cet outil a professionnalisé le projet en automatisant les tâches répétitives (tests, benchmarks, clean), réduisant ainsi les erreurs humaines.

3. Retour d'expérience

3.1. Ingénierie des données et Efficacité

Ce projet nous a appris que la performance d'une IA repose sur la symbiose entre la structure et l'outil :

- **Conception de l'état 11-bits** : Nous avons appris à créer une structure de données adaptée qui compresse l'environnement sans perte d'information critique. C'est ce choix qui a permis une efficacité mémoire optimale (1258 états pour quelques Ko).

- **Maîtrise des flux (MsgPack)** : Le passage au format binaire nous a montré comment choisir le bon outil de traitement pour garantir un temps de calcul $O(1)$ lors de l'accès aux données, rendant l'IA plus rapide que l'affichage graphique.

3.2. Architecture et Robustesse

L'application des principes **SOLID** et du pattern **MVC** a été l'enseignement architectural le plus riche :

- **Modèle fonctionnel au CDC** : Nous avons conçu une architecture où l'IA est une "stratégie" interchangeable. Cela nous a appris à construire des systèmes évolutifs où l'on peut remplacer un algorithme par un autre sans toucher au moteur de jeu.

- **Gestion des contraintes** : L'utilisation de technologies modernes (**Python 3.12, uv, Makefile**) nous a montré comment industrialiser un projet pour le rendre reproductible et professionnel.

3.3. Validation et Qualité Logicielle

La rigueur des tests a transformé notre approche du développement :

- **Tests unitaires et fonctionnels** : La conception de tests sous **Pytest** nous a appris à anticiper les écarts de comportement (collisions, bugs de logique) avant même la phase de benchmark.
- **Documentation utilisateur** : Produire une documentation claire et des commandes automatisées nous a fait comprendre qu'un projet n'est réussi que s'il est utilisable et compréhensible par un tiers.

4. Conclusion

4.1. Synthèse de la Réussite Technique

- **Architecture et Modularité** : L'adoption du pattern MVC et du Design Pattern Strategy a garanti un couplage faible entre les composants. Cette structure a permis une intégration fluide d'algorithmes radicalement différents (A* et Q-Learning) sans altérer l'intégrité du moteur de jeu.
- **Optimisation des Ressources** : La conception d'une structure de données sur 11 bits et l'usage du format de sérialisation binaire MsgPack ont répondu aux contraintes d'efficacité mémoire. Ces choix garantissent une latence décisionnelle minimale $O(1)$ et une portabilité accrue du système.
- **Fiabilité et Convergence** : Les benchmarks, réalisés sous un environnement multithreadé, confirment la stabilité du modèle. L'agent par renforcement a démontré une capacité de convergence réelle, prouvant la pertinence de la fonction de récompense (*Reward Shaping*) implémentée.

4.2. Perspectives et Évolutivité

1. **Transition Technologique (Deep RL)** : Le système est prêt pour l'intégration de réseaux de neurones profonds (DQN via PyTorch), l'interface BaseAgent ayant été conçue pour cette évolution.
2. **Complexification de l'Environnement** : L'introduction du mode Multi-Agent permettrait d'étudier les comportements compétitifs et les protocoles d'évitement de collisions dans un espace partagé.
3. **Industrialisation** : La conteneurisation du projet via Docker et l'automatisation avancée des tests pourraient transformer ce prototype en un outil de benchmark IA standardisé pour l'intégration continue.

Annexes

- Figure 1 : Analyse des métriques via l'interface de monitoring TensorBoard.
- Figure 2 : Interface de simulation multisystème (Mode Duel) sur grille 20x20.

- Figure 3 : Interface d’analyse comparative des performances
- Figure 4 : Analyse comparative des indicateurs clés de performance (KPI) — Agent A vs Agent RL.*
- **Dépôt GitHub officiel** : https://github.com/haMC365/Snake_AI_RL

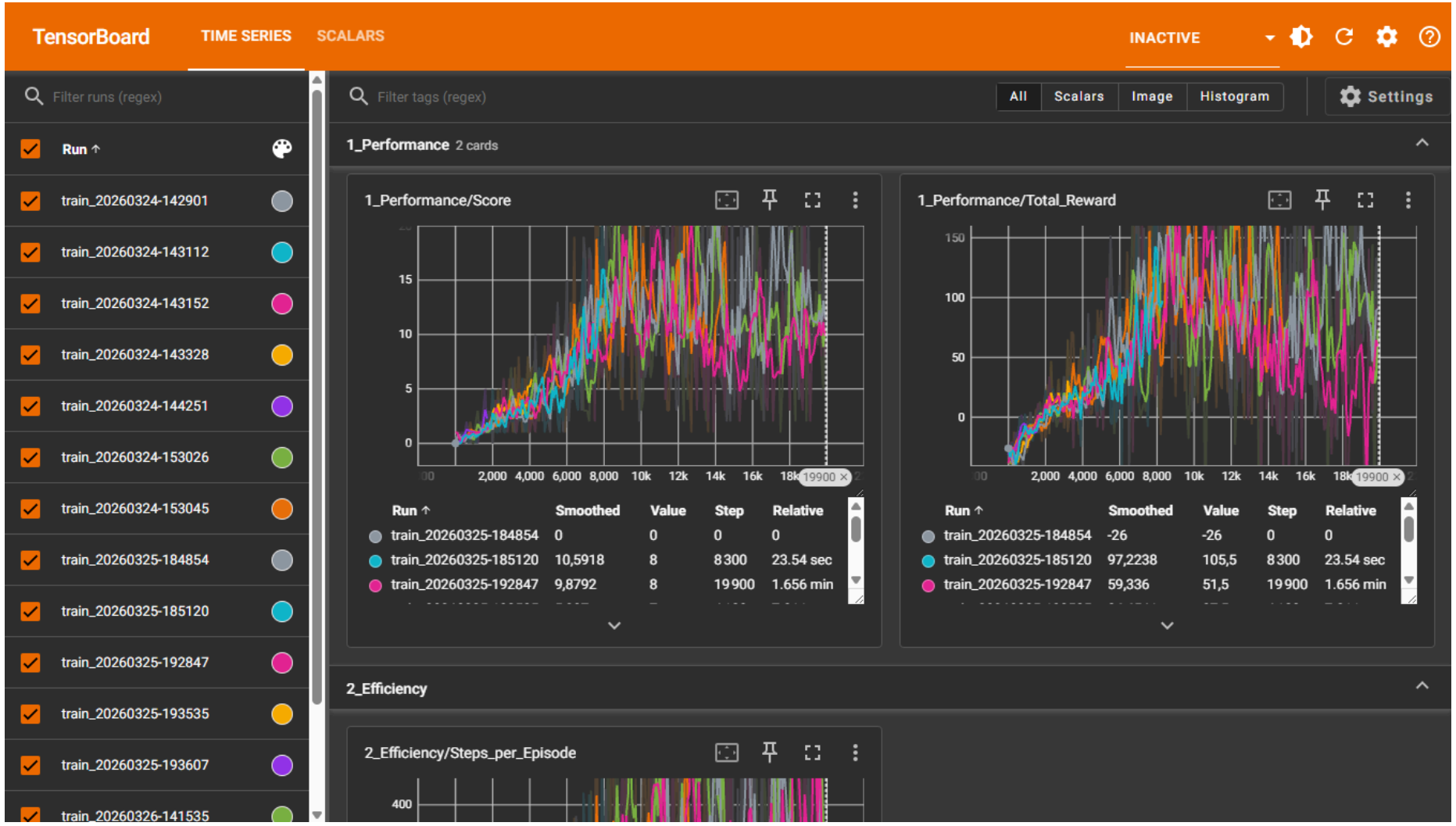

Figure 1 : Analyse des métriques via l'interface de monitoring TensorBoard.

Figure 2 : Interface de simulation multisystème (Mode Duel) sur grille 20x20.

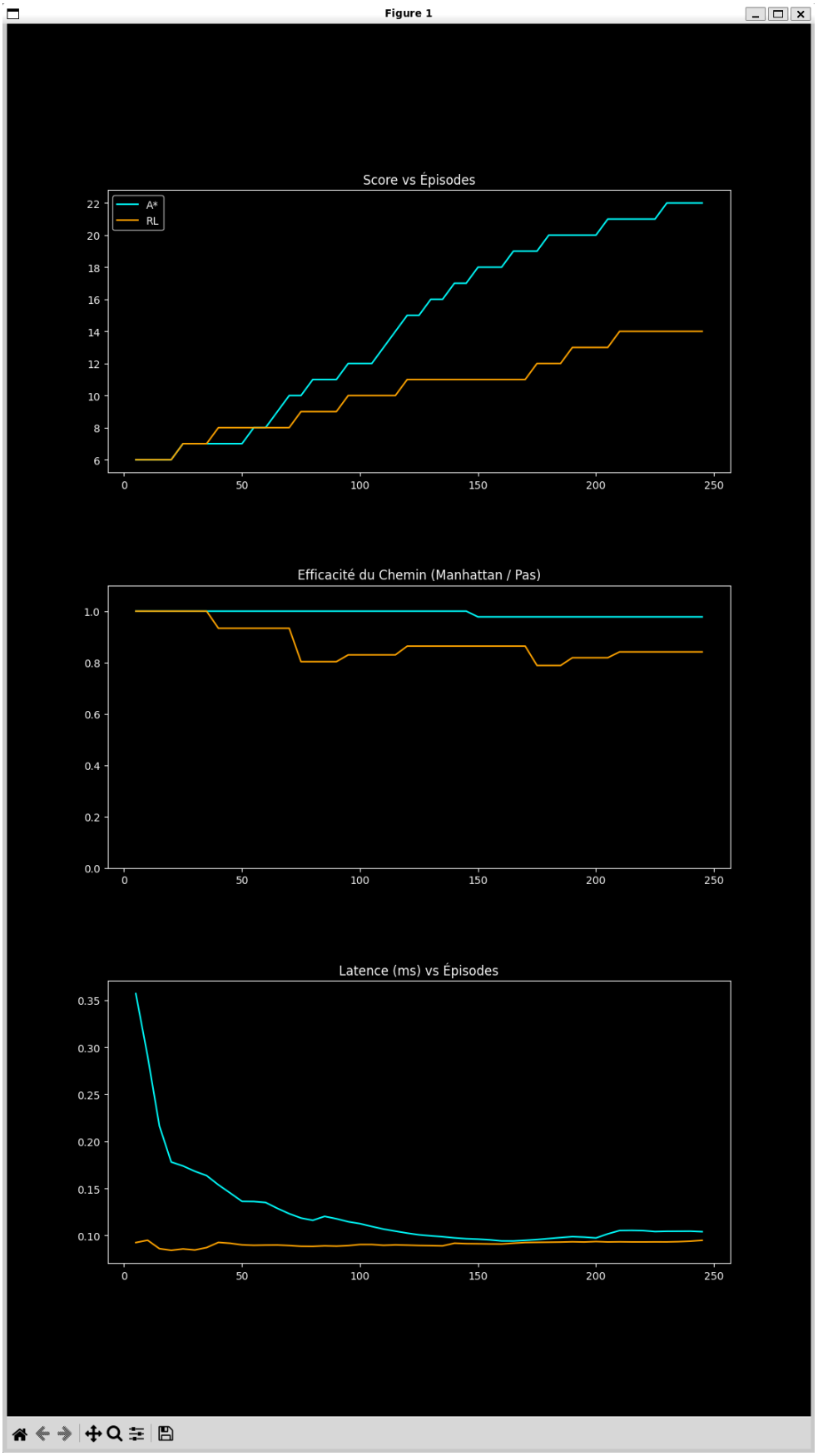

Figure 3 : Interface d'analyse comparative des performances

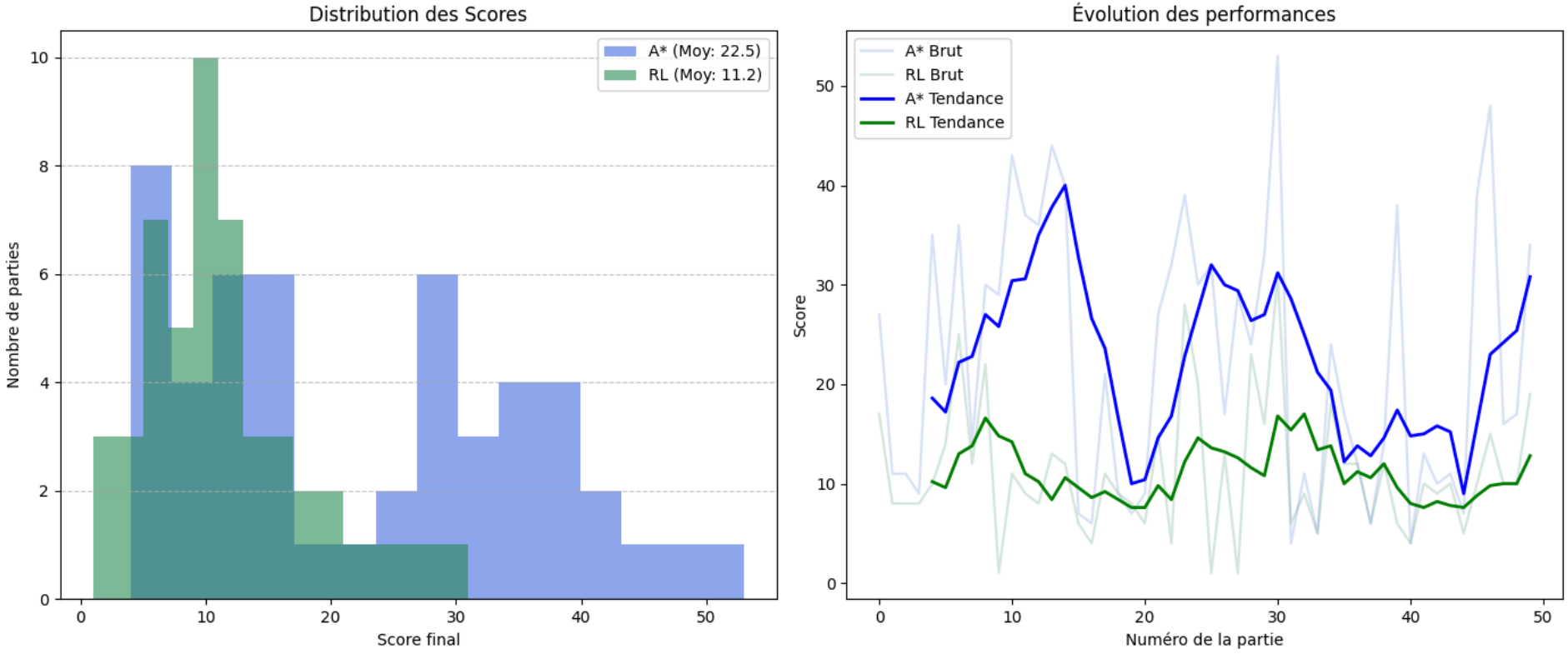

Figure 4 : Analyse comparative des indicateurs clés de performance (KPI) — Agent A vs Agent RL.*